Joel Sivanish

Professor Kaur

Lab 5 - Secure Chat Application with RSA

2/18/2026

Documentation of Recorded Traffic:

1. In your Wireshark capture, locate the packet containing the RSA Public Key (Headers may start with RSA_REQ_PUB). What specific values (E,N) are visible to an observer? Explain why revealing these values does not compromise the private key.

- Visible Values (e, n): An observer can see the Public Exponent (e), which is 65537, and the Modulus (n), which is 162346....883.
- Security Justification: Revealing these values does not compromise the private key because n is the product of two 1024-bit primes. Factoring such a large number to find p and q (and subsequently the private exponent d) is computationally infeasible.

2. Locate a packet containing a secure message (Headers may start with SECURE). Identify the three distinct components of the payload (Signature, Public Key, Ciphertext). Why is the ”Public Key” in this packet different from the one in the handshake?

- Packet Payload: In a SECURE packet, the payload is separated by colons into three parts: Digital Signature, DH Public Key, and Ciphertext.
    - Digital Signature: 165458...986
    - DH Key: 307394...307
    - Ciphertext: d8c05d...63f
- Public Key Distinction: The Public Key seen here is a transient Diffie-Hellman key used for session-specific encryption. This is distinct from the RSA Public Key exchanged earlier, which serves as a long-term identity key for authentication.

3. If an attacker were to capture the RSA_REQ_PUB packet, could they use it to decrypt the messages in the SECURE packets? Explain why or why not, referencing the ”Diffie-Hellman” component of the protocol.

- Attacker Limitation: An attacker capturing the RSA_REQ_PUB packet cannot decrypt secure messages.

- Reasoning: The RSA keypair is used only for signatures (authentication) to verify the source of the message. The actual encryption is performed using AES, with keys derived from a Diffie-Hellman exchange; since the attacker lacks the private DH keys, they cannot derive the session secret.

4. In the SECURE packet, we verify a digital signature. Explain exactly what data is being signed. If an attacker modified the ”Ciphertext” in transit, would the RSA signature verification fail? Why or why not?

- Data Signed: The DH Public Key is the specific data signed by the rsa_sign function.
- Tampering Scenario: If an attacker modifies the Ciphertext in transit, the RSA signature verification would still pass because the signature only covers the public key. However, the message would be rejected during decryption because the HMAC handled in cipher.py would detect the modification of the ciphertext.

5. Compare the ”Plaintext” packets from the start of the chat with the ”Secure” packets. Beyond the unreadable payload, what differences do you see in the packet length or structure?

- Plaintext Packet (Packet 11):
  - Length: 91 bytes (TCP payload of 25 bytes).
  - Content: This corresponds to the first message MSG:ice cream got no legs. It contains only the header and the raw string.
- Secure Packet (Packet 15):
  - Length: 1445 bytes (TCP payload of 1379 bytes).
  - Content: This corresponds to the first SECURE message.
- Structural Difference: Beyond the encrypted payload, the Secure packet is significantly larger (over 1,300 bytes larger). This is because it must carry the 2048-bit RSA Digital Signature and the 2048-bit DH Public Key required for authentication and key exchange.

6. Verify the ”Forward Secrecy” property: In your capture, locate two consecutive SECURE packets. Are the ”Ephemeral DH Public Key” values sent in these packets the same or different?d What does this tell an observer about how the session keys are managed?

- Packet 15 DH Key: 307394…307
- Packet 17 DH Key: 678320…736

- Verification: Even though these messages were sent just seconds apart, the Ephemeral DH Public Keys are completely different. This proves that the application generates a fresh key pair for every single message. This property, known as Forward Secrecy, ensures that if one session key is compromised, an attacker cannot use it to decrypt any other messages in the stream.

7. Based on the Python code you implemented in rsa.py, explain how your rsa_sign function prevents an attacker from forging a message even if they know your public key.

- Mechanism: The rsa_sign function takes the SHA-256 hash of the DH public key and encrypts (signs) it using the sender's Private Key (d, n).
- Logic: An attacker may capture the Public Key (e, n) from the handshake, but they do not have the private exponent (d). Because they lack the private key, they cannot generate a valid signature for a fake message.
- Result: When the recipient receives a packet, they use rsa_verify with the sender's public key. If an attacker attempted to forge a signature, the verification would fail, and the client would discard the packet as unauthentic.